

Problem 1. Intersection of Convex Sets

Show that the intersection of any number of convex sets remains convex.

Hint. Recall the definition of convex sets:

$$x_1, x_2 \in C, 0 \leq \theta \leq 1 \Rightarrow \theta x_1 + (1 - \theta)x_2 \in C.$$

Then apply the properties of intersection. Note that the property holds trivially for empty sets. To see this, note that $A \Rightarrow B$ is the same as “not A or B ”. For empty sets, we have that not A is true since any empty set has no members. Hence, (not A or B) holds.

Solution. Let $\{C_i\}_{i \in I}$ be any collection of convex sets and let

$$C = \bigcap_{i \in I} C_i.$$

Pick any $x_1, x_2 \in C$. Being in the intersection means $x_1, x_2 \in C_i$ for every $i \in I$. Since each C_i is convex, for any $0 \leq \theta \leq 1$ we have

$$\theta x_1 + (1 - \theta)x_2 \in C_i \quad \text{for all } i \in I.$$

So that same point is in every C_i , hence it's in their intersection C . That's exactly the definition of C being convex.

Problem 2. Affine Sets

Show that halfspaces of the form

$$S = \{x \mid a^T x \leq b\}, \quad a \neq 0$$

are convex.

Hint. Use the definition of convex sets and the properties of inequalities for θx_1 and $(1 - \theta)x_2$ where $x_1, x_2 \in S$.

Solution. Let $S = \{x \mid a^T x \leq b\}$ with $a \neq 0$. Take any $x_1, x_2 \in S$. That means

$$a^T x_1 \leq b \quad \text{and} \quad a^T x_2 \leq b.$$

Now take any $0 \leq \theta \leq 1$ and look at the convex combination

$$x = \theta x_1 + (1 - \theta)x_2.$$

Then

$$a^T x = a^T(\theta x_1 + (1 - \theta)x_2) = \theta a^T x_1 + (1 - \theta)a^T x_2 \leq \theta b + (1 - \theta)b = b.$$

So $a^T x \leq b$, which means $x \in S$. Since this works for any two points in S and any $\theta \in [0, 1]$, the halfspace S is convex.

Problem 3. Positive Semidefinite Matrices and Convex Functions

Suppose that $A_1, A_2, \dots, A_m$ are symmetric positive semidefinite.

- (a) Show that $A_1 + A_2 + \dots + A_m$ is symmetric positive semidefinite. In other words, show that symmetric positive semidefinite matrices are closed under addition.

Hint. A matrix A is symmetric positive semidefinite if $x^T A x \geq 0$ for all x . Use this property and inequality properties.

Solution. Let $A_1, A_2, \dots, A_m$ be symmetric positive semidefinite (PSD) matrices. First, since each A_i is symmetric, their sum

$$A = A_1 + A_2 + \dots + A_m$$

is also symmetric.

Now take any vector x . Then

$$x^T A x = x^T (A_1 + \dots + A_m) x = x^T A_1 x + \dots + x^T A_m x.$$

But each A_i is PSD, so $x^T A_i x \geq 0$ for all i . A sum of nonnegative numbers is nonnegative, so

$$x^T A x \geq 0 \quad \text{for all } x.$$

Therefore A is symmetric positive semidefinite.

- (b) Assume that $w_1, w_2, \dots, w_m \geq 0$. Show that

$$w_1 A_1 + w_2 A_2 + \dots + w_m A_m$$

is also symmetric positive semidefinite.

Solution. Let $A_1, \dots, A_m$ be symmetric positive semidefinite and let $w_1, \dots, w_m \geq 0$. Define

$$A = w_1 A_1 + w_2 A_2 + \dots + w_m A_m.$$

Each A_i is symmetric, so any nonnegative scalar multiple $w_i A_i$ is also symmetric, and a sum of symmetric matrices is symmetric; hence A is symmetric.

Now take any vector x . Then

$$x^T A x = x^T (w_1 A_1 + \dots + w_m A_m) x = w_1 x^T A_1 x + \dots + w_m x^T A_m x.$$

Since each A_i is positive semidefinite, $x^T A_i x \geq 0$. Since also $w_i \geq 0$, every term $w_i x^T A_i x \geq 0$. A sum of nonnegative terms is nonnegative, so

$$x^T A x \geq 0 \quad \text{for all } x.$$

Therefore A is symmetric positive semidefinite.

- (c) Suppose that $f_1, f_2, \dots, f_m$ are convex and $w_1, w_2, \dots, w_m \geq 0$. Furthermore, assume that $f_1, f_2, \dots, f_m$ are continuously differentiable up to the second derivative. Show that

$$w_1 f_1(x) + w_2 f_2(x) + \dots + w_m f_m(x)$$

is convex.

Hint. Consider the Hessian of $w_1f_1(x) + \dots + w_mf_m(x)$ and part (b).

Solution. Let $f_1, \dots, f_m$ be convex and let $w_1, \dots, w_m \geq 0$. Define

$$f(x) = w_1f_1(x) + \dots + w_mf_m(x).$$

We show f is convex using the definition.

Take any x, y and any $0 \leq \theta \leq 1$. Since each f_i is convex,

$$f_i(\theta x + (1 - \theta)y) \leq \theta f_i(x) + (1 - \theta)f_i(y).$$

Multiply both sides by $w_i \geq 0$:

$$w_i f_i(\theta x + (1 - \theta)y) \leq w_i \theta f_i(x) + w_i(1 - \theta)f_i(y).$$

Now sum over $i = 1, \dots, m$:

$$\sum_{i=1}^m w_i f_i(\theta x + (1 - \theta)y) \leq \theta \sum_{i=1}^m w_i f_i(x) + (1 - \theta) \sum_{i=1}^m w_i f_i(y).$$

But the left-hand side is $f(\theta x + (1 - \theta)y)$ and the right-hand side is $\theta f(x) + (1 - \theta)f(y)$. So

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y),$$

which is exactly convexity.

- (d) We consider a more direct proof of (c) that does not require that the functions are differentiable. Define $g(x)$ using

$$g(x) = w_1f_1(x) + w_2f_2(x) + \dots + w_mf_m(x)$$

for $w_1, \dots, w_m \geq 0$ and $f_1, \dots, f_m$ convex. Show that $g(x)$ is convex even though it may not be differentiable.

Hint. You will need to use the definition of convexity and add up the inequalities from the individual convex functions to prove this.

Solution. Let $f_1, \dots, f_m$ be convex and $w_1, \dots, w_m \geq 0$. Define

$$g(x) = \sum_{i=1}^m w_i f_i(x).$$

We want to show g is convex without assuming differentiability.

Take any x, y and $0 \leq \theta \leq 1$. Since each f_i is convex, we have

$$f_i(\theta x + (1 - \theta)y) \leq \theta f_i(x) + (1 - \theta)f_i(y).$$

Multiply by $w_i \geq 0$:

$$w_i f_i(\theta x + (1 - \theta)y) \leq \theta w_i f_i(x) + (1 - \theta)w_i f_i(y).$$

Now sum these inequalities over $i = 1, \dots, m$:

$$\sum_{i=1}^m w_i f_i(\theta x + (1 - \theta)y) \leq \theta \sum_{i=1}^m w_i f_i(x) + (1 - \theta) \sum_{i=1}^m w_i f_i(y).$$

The left side is $g(\theta x + (1 - \theta)y)$ and the right side is $\theta g(x) + (1 - \theta)g(y)$. So

$$g(\theta x + (1 - \theta)y) \leq \theta g(x) + (1 - \theta)g(y),$$

which is exactly the definition of convexity. Thus g is convex even if the f_i 's are not differentiable.

(e) Based on (d) show that the elastic net formulation is convex:

$$\|y - X\beta\|_2^2 + \lambda_2 \|\beta\|_2^2 + \lambda_1 \|\beta\|_1$$

where y, β are vectors, X is a matrix, and $\lambda_1, \lambda_2 > 0$.

Hint. From slide 3-4, we have the following convex norms: $\|\beta\|_p$ for $p \geq 1$ which includes $p = \infty$. In addition, evaluate the Hessians of

$$\|y - X\beta\|_2^2 = (y - X\beta)^T(y - X\beta), \quad \|\beta\|_2^2 = \sum_i \beta_i^2 = \beta^T \beta$$

and show that they are symmetric positive semidefinite based on 5(b).

Solution. We want to show that

$$F(\beta) = \|y - X\beta\|_2^2 + \lambda_2 \|\beta\|_2^2 + \lambda_1 \|\beta\|_1$$

is convex for $\lambda_1, \lambda_2 > 0$.

1. The term $\|y - X\beta\|_2^2 = (y - X\beta)^T(y - X\beta)$ is a quadratic function in β . Its Hessian is $2X^T X$, and $X^T X$ is symmetric positive semidefinite, so this term is convex.

2. The term $\|\beta\|_2^2 = \beta^T \beta$ has Hessian $2I$, which is symmetric positive definite, so $\|\beta\|_2^2$ is convex. Multiplying by $\lambda_2 > 0$ keeps it convex.

3. The term $\|\beta\|_1$ is a norm (with $p = 1$), and norms are convex. Multiplying by $\lambda_1 > 0$ keeps it convex.

Now we have a sum of convex functions, each with a nonnegative weight. By the result in (d), the sum is convex. Therefore, the elastic net objective is convex.

Problem 4. Convex Functions in 1D

- (a) Use the definition of convexity and concavity to show that
- $ax+b$
- is both convex and concave.

Solution. A function f is convex if

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \quad \forall x_1, x_2 \in \mathbb{R}, t \in [0, 1].$$

Let $f(x) = ax + b$. Then

$$f(tx_1 + (1-t)x_2) = a(tx_1 + (1-t)x_2) + b = t(ax_1 + b) + (1-t)(ax_2 + b) = tf(x_1) + (1-t)f(x_2).$$

Thus equality holds, implying both

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \quad \text{and} \quad f(tx_1 + (1-t)x_2) \geq tf(x_1) + (1-t)f(x_2).$$

Hence, $f(x) = ax + b$ is both convex and concave (i.e., affine).

- (b) Use the second derivative test to establish that the following functions are convex:

$$e^{ax} \text{ for } x \in \mathbb{R}, \quad x^\alpha \text{ for } x \in \mathbb{R}_{++} \text{ and } \alpha \geq 1, \quad x \log x \text{ for } x \in \mathbb{R}_{++}.$$

Solution. Recall: if f is twice differentiable on an interval and $f''(x) \geq 0$ for all x in that interval, then f is convex there.

1. For
- $f(x) = e^{ax}$
- , we have

$$f'(x) = ae^{ax}, \quad f''(x) = a^2e^{ax}.$$

Since $e^{ax} > 0$ and $a^2 \geq 0$, we get $f''(x) \geq 0$ for all $x \in \mathbb{R}$, so e^{ax} is convex on $\mathbb{R}$.

2. For
- $f(x) = x^\alpha$
- with
- $x \in \mathbb{R}_{++}$
- and
- $\alpha \geq 1$
- ,

$$f'(x) = \alpha x^{\alpha-1}, \quad f''(x) = \alpha(\alpha-1)x^{\alpha-2}.$$

Here $\alpha(\alpha-1) \geq 0$ and $x^{\alpha-2} > 0$ for $x > 0$, so $f''(x) \geq 0$ on $\mathbb{R}_{++}$, hence x^α is convex on $\mathbb{R}_{++}$.

3. For
- $f(x) = x \log x$
- on
- $\mathbb{R}_{++}$
- ,

$$f'(x) = \log x + 1, \quad f''(x) = \frac{1}{x}.$$

Since $\frac{1}{x} > 0$ for $x > 0$, we have $f''(x) \geq 0$ on $\mathbb{R}_{++}$, so $x \log x$ is convex on $\mathbb{R}_{++}$.

Problem 5. Symmetric Positive Semidefinite Matrices

- (a) Suppose that S is symmetric with positive entries. Show that S is symmetric positive semidefinite for vectors $x \geq 0$.

Hint. Use the definition from 3(a), inequality properties, and the fact that

$$x^T S x = \sum_i \sum_j s_{ij} x_i x_j.$$

Solution. Let $S = [s_{ij}]$ be a symmetric matrix with positive entries, i.e. $s_{ij} > 0$ for all i, j . Take any vector $x \geq 0$, meaning $x_i \geq 0$ for all i . Then

$$x^T S x = \sum_i \sum_j s_{ij} x_i x_j.$$

Since $s_{ij} > 0$ and $x_i, x_j \geq 0$, each term $s_{ij} x_i x_j \geq 0$. Therefore,

$$x^T S x = \sum_i \sum_j s_{ij} x_i x_j \geq 0.$$

Because S is symmetric and $x^T S x \geq 0$ for all $x \geq 0$, we conclude that S is **symmetric positive semidefinite for vectors** $x \geq 0$.

- (b) Suppose that S is symmetric and expressed as $S = VV^T$ for some vector V . Show that S is symmetric positive semidefinite.

Hint. Use the fact that

$$x^T VV^T x = (V^T x)^T (V^T x).$$

Solution. Let $S = VV^T$ for some (column) vector $V \in \mathbb{R}^n$. First, S is symmetric because

$$S^T = (VV^T)^T = VV^T = S.$$

Now take any $x \in \mathbb{R}^n$. Then

$$x^T S x = x^T (VV^T) x = (V^T x)^T (V^T x).$$

But $(V^T x)$ is a scalar, so

$$(V^T x)^T (V^T x) = (V^T x)^2 \geq 0.$$

Thus $x^T S x \geq 0$ for all x , and since S is symmetric, S is symmetric positive semidefinite.

Problem 6. Multiobjective Optimization I (This problem is worth twice as much credit as the other problems.)

- (a) Suppose that $f_1(\cdot)$ and $f_2(\cdot)$ represent two objectives that we are trying to minimize. When comparing points, we say that x is *at least as good as* y in the multiobjective sense if

$$f_1(x) \leq f_1(y) \quad \text{and} \quad f_2(x) \leq f_2(y).$$

Usually, there is no one point that is better than all other points. Instead, in multiobjective optimization (or vector optimization), we compute the Pareto front as the set of points that cannot be improved upon by any other point.

More formally, x is a Pareto point if we cannot find another point that matches x in all but at least one objective:

$$x \text{ is Pareto-optimal} \Rightarrow \neg(\exists y \exists j \forall i \neq j \ f_j(x) > f_j(y) \text{ and } f_i(y) \leq f_i(x)).$$

Negate this statement to show that:

$$x \text{ is Pareto-optimal} \Rightarrow \forall y \forall j \text{ except for index } i : f_j(x) \leq f_j(y) \text{ or } f_i(y) > f_i(x).$$

In other words, a point is Pareto optimal if no other point can be found that is at least as good in all criteria.

Hint. We propagate the negation from left to right. Also note that $\neg\forall y$ becomes $\exists y$ and $\neg\exists y$ becomes $\forall y$. Then the negation of (A and B) is $(\neg A \text{ or } \neg B)$.

Solution. We begin with the definition of a Pareto-optimal point:

$$x \text{ is Pareto-optimal} \Rightarrow \neg(\exists y \exists j \forall i \neq j \ [f_j(x) > f_j(y) \text{ and } f_i(y) \leq f_i(x)]).$$

This expression says that there does not exist another point y and an objective j such that y performs strictly better than x in objective j and at least as well in all others.

To obtain the equivalent positive form, we move the negation inward step by step, using the standard logical rules:

$$\neg(\exists) = \forall, \quad \neg(\forall) = \exists, \quad \neg(A \wedge B) = (\neg A) \vee (\neg B).$$

Applying these in sequence:

$$\begin{aligned} & \neg(\exists y \exists j \forall i \neq j \ [f_j(x) > f_j(y) \text{ and } f_i(y) \leq f_i(x)]) \\ &= \forall y \neg(\exists j \forall i \neq j \ [f_j(x) > f_j(y) \text{ and } f_i(y) \leq f_i(x)]) \\ &= \forall y \forall j \neg(\forall i \neq j \ [f_j(x) > f_j(y) \text{ and } f_i(y) \leq f_i(x)]) \\ &= \forall y \forall j \exists i \neq j \ \neg(f_j(x) > f_j(y) \text{ and } f_i(y) \leq f_i(x)) \\ &= \forall y \forall j \exists i \neq j \ (f_j(x) \leq f_j(y) \text{ or } f_i(y) > f_i(x)). \end{aligned}$$

Therefore, we can express Pareto optimality equivalently as:

$$x \text{ is Pareto-optimal} \implies \forall y \forall j \exists i \neq j : (f_j(x) \leq f_j(y) \text{ or } f_i(y) > f_i(x)).$$

In words, this means that for every other point y and every objective j , there is at least one objective (possibly a different one) where either x performs no worse than y , or y performs worse than x . Thus, no point y exists that is at least as good as x in all objectives and strictly better in one.

- (b) Suppose that we are looking at accuracy and computational time. As before, we want to minimize the computational time objective. However, we want to have higher accuracy for this objective. Modify the code so that the new front determines that x is better than y if

$$(\text{accuracy}(x) \geq \text{accuracy}(y) \text{ and } \text{time}(x) < \text{time}(y))$$

or

$$(\text{accuracy}(x) > \text{accuracy}(y) \text{ and } \text{time}(x) \leq \text{time}(y)).$$

Modify the provided code so that it computes the new front and produce a plot of accuracy (vertical axis) against time (horizontal axis) and demonstrate your function works against 10 random points.

Note. Do not oversample. There is one point that is optimal and better than all of the others. Can you see which one? If you sample only a few points, you should be getting a nice front.

Solution. Here we're minimizing computational time and maximizing accuracy. A point is considered better if it's faster while maintaining equal or higher accuracy, or if it's more accurate at the same or lower time. Using 10 random samples, the figure below shows the resulting Pareto front with accuracy on the vertical axis and time on the horizontal axis.

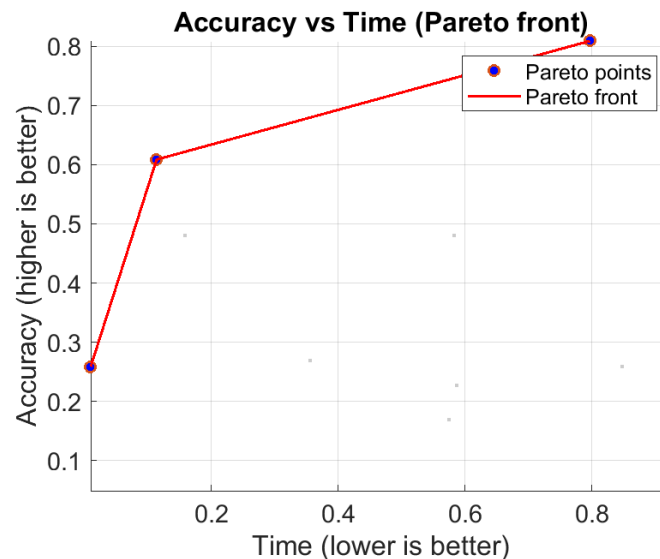


Figure 1: Accuracy vs. Time for 10 random samples. Blue dots are Pareto-optimal points; the red line shows the Pareto front.

The plot makes the trade-off clear—higher accuracy generally comes with longer computation time. The top-left point stands out as the best overall, since it's both accurate and fast.

Problem 7. Multiobjective Optimization II (This problem is worth twice as much credit as the other problems.)

- (a) Suppose that the Pareto front is given by

$$f_2(x) = \frac{1}{f_1(x)} \quad \text{for } x > 0.$$

First, we need to show that this is a valid relationship. To verify that this equation can represent a Pareto front, consider any two distinct points of the form $(f_1, 1/f_1)$ and show that one cannot dominate the other.

Solution. The given front is

$$f_2 = \frac{1}{f_1}, \quad f_1 > 0.$$

To check if this can represent a valid Pareto front, take two different points on the curve:

$$P = (f_1, 1/f_1), \quad Q = (g_1, 1/g_1),$$

where $f_1 \neq g_1$ and both are positive.

If $f_1 < g_1$, then P has a smaller f_1 (better in that objective), but a larger f_2 (worse in the other). If $f_1 > g_1$, the opposite happens— Q is better in f_1 but worse in f_2 .

In both cases, neither point dominates the other. This means no point along the curve can be strictly better in both objectives, which is exactly what defines a Pareto front. So, $f_2 = 1/f_1$ for $f_1 > 0$ is indeed a valid Pareto front.

- (b) Show that the relationship in 7(a) is both one-to-one and convex. To show that the equation is one-to-one, simply show that we cannot have two different values of f_1 map to the same $1/f_1$.

Solution. The relationship is

$$f_2 = \frac{1}{f_1}, \quad f_1 > 0.$$

It's one-to-one because no two different values of f_1 can produce the same f_2 . If $f_{1a} \neq f_{1b}$, then $1/f_{1a} \neq 1/f_{1b}$, since the reciprocal function is strictly decreasing for positive values.

To check convexity, take the second derivative of f_2 with respect to f_1 :

$$\frac{df_2}{df_1} = -\frac{1}{f_1^2}, \quad \frac{d^2 f_2}{df_1^2} = \frac{2}{f_1^3}.$$

Because $f_1 > 0$, the second derivative is positive, so the curve is convex. Hence, the relationship $f_2 = 1/f_1$ is both one-to-one and convex.

- (c) Suppose that we want to use scalarization to find the points on the Pareto front. In this case, we would minimize

$$f_1(x) + \lambda f_2(x).$$

Take the gradient of this expression and set it to zero to find a relationship between ∇f_1 and ∇f_2 .

Solution. Using scalarization, we minimize the combined objective

$$f_1(x) + \lambda f_2(x),$$

where $\lambda > 0$ is a weighting factor.

Taking the gradient and setting it equal to zero gives

$$\nabla f_1(x) + \lambda \nabla f_2(x) = 0.$$

Rearranging, we get the relationship

$$\nabla f_1(x) = -\lambda \nabla f_2(x).$$

This shows that, at any Pareto-optimal point, the gradients of f_1 and f_2 are opposite in direction and scaled by λ .

- (d) Consider the tangent lines on the Pareto front. The slope at each point is given by $\frac{df_2}{df_1}$. As discussed in the book, the vector $(1, \lambda)$ is orthogonal to the local tangent line. Explain how you can select λ to get different points on the Pareto front. Provide an example.

Solution. The slope of the Pareto front is given by

$$\frac{df_2}{df_1},$$

and the vector $(1, \lambda)$ is orthogonal to this tangent line. Because orthogonality implies

$$\frac{df_2}{df_1} = -\frac{1}{\lambda},$$

changing the value of λ changes the slope of the tangent line and, in turn, selects different points along the front.

For example, using the front $f_2 = 1/f_1$, we have

$$\frac{df_2}{df_1} = -\frac{1}{f_1^2}.$$

Comparing with $-1/\lambda$, we find

$$\lambda = f_1^2.$$

So, by varying λ , we can move to different f_1 (and corresponding f_2) values on the Pareto front — small λ emphasizes minimizing f_1 , while large λ emphasizes minimizing f_2 .

- (e) Note that we can also use the Pareto front to solve constrained optimization problems. Consider the following problem:

$$\min_x f_2(x) \quad \text{for } f_1 \leq b.$$

Show that the solution is on the boundary. Repeat for:

$$\min_x f_1(x) \quad \text{for } f_2 \leq b.$$

Solution. Think of the Pareto front as giving us the “best you can do” in one objective for any fixed value of the other. So if we want to

$$\min_x f_2(x) \quad \text{subject to } f_1(x) \leq b,$$

then among all points whose f_1 is at most b , the one with the smallest f_2 will lie on the Pareto front at exactly $f_1 = b$. If it were at some $f_1 < b$, then (by moving along the front to the right, up to $f_1 = b$) we could often do better in f_2 , so the optimal feasible point gets “pushed” to the boundary.

By the same logic, for

$$\min_x f_1(x) \quad \text{subject to } f_2(x) \leq b,$$

the best point will lie on the part of the Pareto front where $f_2 = b$. Going below the boundary in f_2 would force us off the front and give a worse f_1 , so again the solution is on the constraint boundary.

Provided MATLAB Code

You may include the provided code as follows:

```

1 % Regular multi-objective optimization methods:
2 % Regular methods assume that objective values are available over
3 % regularly spaced intervals. This is a trivial case and it is one for
4 % which all of the problems can be solved directly.
5 clear
6 clc
7 run_reg_mo_obj
8
9 function run_reg_mo_obj
10 N = 10;
11 M = 10;
12 fs = 14; % font size
13 lw = 1.5; % line width
14 mo2d = setup_multi_obj_rand2d(N, M);
15 mo2d_pareto_points = pareto_opt2d(mo2d);
16 mo2d_pareto_front = pareto_front_conts2d(mo2d);
17 plot_mo_all(1, mo2d, mo2d_pareto_points, mo2d_pareto_front, fs, lw);
18 end
19
20 % -----
21 % A regular structure constructor to guarantee that
22 % we have uniform structure names.
23 function [mo_struct] = make_mo_struct (x1, x2, o1, o2)
24 mo_struct = struct('x1', x1, 'x2', x2, 'o1', o1, 'o2', o2);
25 end
26
27 % -----
28 % setup_multi_obj_rand2d():
29 % Sets up all of the objective using random functions
30 % In this special structure "multi_obj2d", arrays
31 % are created using meshgrid() and stored regularly.
32 function [mo2d] = setup_multi_obj_rand2d(N, M)
33
34 % Uniformly sample the parameter space
35 x1_range = linspace(0, 1, N);
36 x2_range = linspace(0, 1, M);
37
38 % Create a uniform grid of the parameters space based on the range:
39 [x1, x2] = meshgrid(x1_range, x2_range);
40
41 % For each one of the parameter values create random values:
42 % (assignment wants 10 random points, so just make 1-by-N arrays)
43 f1 = rand(1, N); % time (to minimize)
44 f2 = rand(1, N); % accuracy (to maximize)
45 x1 = rand(1, N);
46 x2 = rand(1, N);
47
48 % Put everything in a nice multi-objective optimization structure:
49 mo2d = make_mo_struct(x1, x2, f1, f2);
50 end
51

```

```

52 % -----
53 % pareto_opt2d()
54 %   Compute the Pareto-optimal points in 2d.
55 %   Store the results in a parallel structure of 1-D arrays.
56 function [mo2d_pareto_points] = pareto_opt2d(mo2d)
57
58 % Initialize the results:
59 x1 = [];
60 x2 = [];
61 o1 = [];
62 o2 = [];
63
64 % Go through a loop all possible points to identify optimal cases
65 [N, M] = size(mo2d.x1);
66 for i=1:N
67     for j=1:M
68         o1_val = mo2d.o1 (i,j); % time
69         o2_val = mo2d.o2 (i,j); % accuracy
70
71         % Compute the number of non-zero elements that are better using
72         % nnz(.)
73         opt_size = nnz((mo2d.o2 >= o2_val) .* (mo2d.o1 < o1_val)) + ...
74         nnz((mo2d.o2 > o2_val) .* (mo2d.o1 <= o1_val));
75
76         % Nobody can beat this point. So it is optimal:
77         if (opt_size == 0)
78             x1 = [x1, mo2d.x1(i,j)];
79             x2 = [x2, mo2d.x2(i,j)];
80             o1 = [o1, o1_val];
81             o2 = [o2, o2_val];
82         end
83     end
84 end
85
86 % Create the return structure:
87 mo2d_pareto_points = make_mo_struct(x1, x2, o1, o2);
88 end
89
90 % -----
91 % pareto_front2d()
92 %   Generates a Pareto front by computing the Pareto points
93 %   and then sorting through them in the right order.
94 function [mo2d_pareto_front] = pareto_front_conts2d(mo2d)
95
96 % Generate the optimal points:
97 [mo2d_pareto_points] = pareto_opt2d(mo2d);
98
99 % Sort through the data points:
100 % Start from the highest value of o1 and the lowest of o2.
101 % First, re-index based on a descending order from o1.
102 [o1_sorted, o1_ind] = sort(mo2d_pareto_points.o1, 'ascend');
103 o2_vals = mo2d_pareto_points.o2 (o1_ind);
104 x1_vals = mo2d_pareto_points.x1 (o1_ind);
105 x2_vals = mo2d_pareto_points.x2 (o1_ind);

```

```

106
107 % Assuming that the objectives are continuous, we do not need to do
108 % anything else!
109
110 % Store the contents in a structure:
111 mo2d_pareto_front = make_mo_struct(x1_vals, x2_vals, o1_sorted, o2_vals);
112 end
113
114 % -----
115 % plot_mo_all():
116 %   Plots the realization space and Pareto optimal front for visualization.
117 function plot_mo_all (figure_num, mo_all, mo_points, mo_pareto_front, fs, lw)
118 figure(figure_num);
119 clf
120 hold on
121 title('Accuracy vs Time (Pareto front)', 'FontName', 'Arial', 'FontSize', fs);
122 xlabel('Time (lower is better)', 'FontName', 'Arial', 'FontSize', fs);
123 ylabel('Accuracy (higher is better)', 'FontName', 'Arial', 'FontSize', fs);
124 grid on
125 axis tight
126 plot(mo_all.o1(:), mo_all.o2(:), '.', 'Color', [0.8 0.8 0.8], 'HandleVisibility', 'off');
127 h1 = plot(mo_points.o1(:), mo_points.o2(:), 'o', 'MarkerFaceColor', 'b', ...
128 'DisplayName', 'Pareto points', 'LineWidth', lw);
129 h2 = plot(mo_pareto_front.o1(:), mo_pareto_front.o2(:), 'r-', ...
130 'LineWidth', lw, 'DisplayName', 'Pareto front');
131 legend([h1 h2], 'Pareto points', 'Pareto front', 'Location', 'northeast', ...
132 'FontName', 'Arial', 'FontSize', fs-2);
133 set(gca, 'FontName', 'Arial', 'FontSize', fs);
134 hold off
135 saveas(gcf, 'fig_1.png');
136 end
137

```